

TrainAudit: Silent Error Detection for LLM Training via Verified Guarded Relational Constraints—Supplementary Material

Qingsong Cai*
Sichuan University
Chengdu, China
qingsong1010@gmail.com

Shikai Li*
West China Hospital, Sichuan
University
Chengdu, China
lishikai@wchscu.edu.cn

Zhengmao Ye
Sichuan University
Chengdu, China
yehengmaolove@gmail.com

Mingjie Tang
Sichuan University
Chengdu, China
tangrock@gmail.com

Ran Tao
iQuest Lab
Beijing, China
rtao02@iquestlab.com

Bryan Dai
iQuest Lab
Beijing, China
cbdai@iquestlab.com

ABSTRACT

This PDF contains supplementary material for the accompanying PVLDB submission. The main paper is self-contained; this supplement provides the methodology protocols, mining and SQL specifications, and per-case tables needed to audit the main-text claims. Prompt dumps and collector scripts are provided with the artifact.

*Qingsong Cai and Shikai Li contributed equally to this work.

ROADMAP: APPENDIX SECTIONS VS. MAIN PAPER

Table .1 maps each appendix section to the main-paper claim it supports. Primary reading order for verifying main claims: B→D→E→F→I (then G/H only if needed). Implementation dumps and figures that only restate the main text are omitted (released with the artifact).

Table .1: Where to look when checking a main-paper claim.

App.	Supports (main)	Content
A	§Related Work	Compact oracle/scope positioning (no new claims)
B	§Empirical / Real-SE setup	Pool construction, silent filter, IRR, reproducibility
C	Table 1 taxonomy	Fine-grained 392-case distribution figure
D	§Pattern Catalog	Frozen induction protocol; matched mining ablation; 92.3% locked test
E	§Invariant Miner	Funnel gates, CE I/O, provenance (SwitchMLP pointer)
F	§Online Detection	Trace schema and compiled SQL
G	Diagnosis paragraph	Structured reports for the 17 detected cases
H	Out-of-pool / production	Production Cases 1–2 (main-text anchors only)
I	§Detection / Efficiency	Per-case table, tier curve, $\mathcal{P}$ deployment, panorama

A EXTENDED RELATED WORK COMPARISON

This note expands one axis of the main-text Related Work: the *oracle source* of correctness. Outcome-metric monitors fire only after macro curves bend; TrainCheck [4] learns single-process API regularities; TTrace [3] and TrainVerify [5] need a reference or a static plan; data-quality validators (CFDs [2], denial constraints [1], Auto-Validate [7], DeeQu [6]) target static tables. TRAINAUDIT inherits the relational-constraint view but binds each obligation to live traces with topology and phase guards π_{topo} and π_{precond} —the distinction already stated in the main text, not a new claim.

B BUG POOL CONSTRUCTION AND METHODOLOGY

This appendix documents the source, filtering, classification, and inter-rater validation of the 392-record silent-error evidence corpus. The merged manifest, deterministic mapping tables, disagreement sets, provenance audit, and the available reproduction artifacts are released with the paper.

B.1 Pool Construction

Four frameworks with auditable training-critical history: Megatron-LM, DeepSpeed, OLMo, and OLMo-core. The corpus frozen on 2026-05-10 merges a 295-record commit-oriented pool and a 128-record issue/PR pool (32 overlaps): retain 96 unique issue/PR records + all 295 + one fully specified orphan $\Rightarrow$ **392 records** (Megatron-LM 110, DeepSpeed 123, OLMo 77, OLMo-core 82). Commit diffs are the preferred experimental unit (concrete buggy/fixed transition); the 96 issue/PR-only records stay for catalog coverage but never enter Real-SE without a later commit-pair audit. Screening keeps training-correctness semantic failures that can proceed without an immediate exception and that expose extractable objects/relation/witness; duplicate upstream incidents are collapsed. Provenance: 289 fixed-commit IDs (288 resolve locally; dates 2021-08-06–2026-05-06), 96 URL-only, 7 summary-only. This is *not* an exhaustive default-branch census; full audit in `benchmark/eval/corpus_provenance_audit.md`. Table B.1 makes the dataset separation explicit: catalog induction and the detector benchmark apply different filters to the same 392-record evidence corpus.

Table B.1: Separation of the three experimental datasets. The catalog study and the detector benchmark apply different filters to the same evidence corpus.

Stage	Input	Construction rule	Output
Evidence corpus	295-record pool + 96 records unique to the 128-record pool + 1 orphan	Retain each of the 32 shared incidents once; exclude an incomplete external orphan before assembly	392 records
Catalog induction	392 records	Incident-grouped, framework/class-stratified seed/dev/test split; Real-SE forced into locked test	128 / 186 / 78
Detector benchmark	392 records	Class/framework coverage targets plus commit pair, runnable driver, archived buggy/fixed evidence, and no surrogate	17 + 1 boundary

B.2 Silent-vs-Loud Filtering

Main-text silent-error criteria cannot all be physically replayed on 392 records, so we separate evidence strength (Table B.2): keyword/metadata triage versus runnable buggy/fixed comparisons. L1 is physical (reproduced, no exception, expected_output differs); L2–L3 are textual;

L4 is a sparse-field review queue. Absence of a newly added raise/assert is supporting counter-evidence only (0 recorded additions); a crash-keyword scan yields 2/392 warnings retained after adjudication (both false positives). Physical anchors are the 23 L1 reproductions, audited Real-SE pairs, and fault replays—not the sum of textual tiers. Per-record tiers: `silent_evidence_392.json`.

Table B.2: Distribution of the 4-tier silent-evidence rating on the 392-record corpus. L1 provides physical buggy/fixed evidence; L2–L3 are textual evidence levels; L4 records have sparse evidence and are retained only as a review queue. Source: `silent_evidence_392.json`.

Tier	Criterion	Count	Note
L1	reproduced + expected_output shows silent	23	physical ground truth
L2	root_cause has silent keywords, no crash keywords	161	—
L3	only issue title carries silent signal	72	—
L4	sparse fields	136	weakest tier; manual review required
<i>Loud-bug counter-evidence</i>			
	available metadata signals a newly added raise/assert	0 recorded	supporting evidence only
	crash-keyword WARNING (retained after manual review)	2/392 (0.5%)	both false positives

B.3 Descriptive Subsystem Coding

Open coding then axial clustering yields 13 coarse reporting labels (*numerical, control_flow, gradient_sync, checkpoint, moe, optimizer_state, data_loading, communication, dtype, loss_computation, sharding, lr_schedule, offload*)—descriptive bins, not the Pattern Catalog. Of 295 records with legacy labels: **138/295** direct match, **154/295** via deterministic ALIAS_MAP, **3/295** adjudicated. Map and scripts: `taxonomy_methodology.md`, `build_392_catalog.py`.

B.4 Inter-Rater Reliability

We report two complementary agreement checks. Protocol A measures re-annotation agreement on 32 doubly-and-independently annotated records; protocol B uses a separate LLM rerater on a 50-record stratified sample and reports Cohen’s κ .

Protocol A: Re-annotation agreement. 32 records in the pool went through two mutually independent annotation flows, forming a natural inter-rater test set. We observe **25/32 = 78.1% raw agreement**; all 7 disagreements are released with per-record rationale, and the disagreements concentrate between the broad *control_flow* label and more specific subsystem labels (*lr_schedule, gradient_sync*).

Protocol B: Independent LLM rerater. We drew a 50-record sample stratified over the reporting labels, hid the reference labels, and had an independent LLM rerater assign labels based only on the issue title, root_cause, and fix diff. Raw agreement is 60.0% (30/50); under the chance baseline of $p_{\text{expected}} = 0.078$ for the observed label vocabulary, this gives **Cohen’s $\kappa = 0.566$** . This moderate agreement reinforces why the labels are used only for coarse corpus description rather than as the detector’s semantic representation. `irr_50_results.json` releases the stripped input, independent and reference labels, per-label metrics, and all disagreement details. Table B.3 summarizes the two protocols.

Table B.3: Inter-rater reliability across the two protocols. Protocol A measures re-annotation agreement on 32 doubly-and-independently annotated bugs; Protocol B uses an independent LLM rerater on a 50-bug stratified sample to compute Cohen’s κ . Source: `irr_50_results.json`.

Protocol	Sample	Metric	Value	Interpretation
A (Re-annotation)	32 bugs	raw agreement	78.1% (25/32)	—
B (Independent LLM)	50 stratified	Cohen’s κ	0.566	moderate (single-round)
B	50 stratified	raw agreement	60.0% (30/50)	—
B	50 stratified	p_{expected}	0.078	label-vocabulary baseline

Label-boundary notes. Protocol B confusions concentrate on broad *control_flow* (per-label F1=0) and symptom-biased *gradient_sync* over-prediction; five labels reach F1 $\geq$ 0.80. Details in `irr_50_results.json`.

B.5 From the Evidence Corpus to Real-SE

Real-SE is not a random sample of reproduction_status. Diversity targets (labels $\times$ frameworks) plus six admission criteria: (i) auditable provenance; (ii) silent semantic failure; (iii) buggy/fixed commit pair; (iv) runnable driver; (v) archived commands/env/logs/verdict; (vi) no pattern-only surrogate. Criteria and manifest are frozen before detector scoring. Result: 17 detector-coverable paired reproductions of

real buggy and fixed revisions at method level (every label; not end-to-end jobs) + boundary O-003 (deliberate miss; in buggy denominator). O-022 stays out (no commit pair) but remains survey/catalog evidence. Detection rate is not a prevalence estimate. Manifest: benchmark/eval/real_sdc/; rebuild build_392_catalog.py/build_manifest_v2.py (392/392; 7/32 disagreements; $\kappa=0.566$).

C FINE-GRAINED DISTRIBUTION OF THE 392-RECORD EVIDENCE CORPUS

The main text gives a compact distribution over the coarse primary subsystem labels. This appendix presents the finer distribution—both by reporting label and by training parallelism dimension—to facilitate experimental reproduction and inspect sampling diversity.

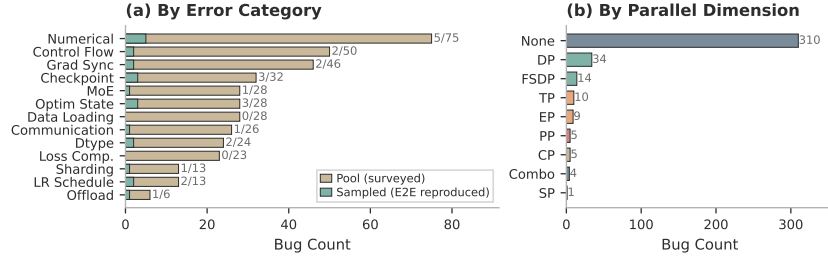


Figure C.1: Fine-grained distribution of the 392-record evidence corpus. (a) By descriptive subsystem label: the background color (sand) marks the corpus, while the foreground (teal) marks historical pre-filter diversity targets used during Real-SE construction; s/p denotes target/pool. (b) By dominant training parallelism dimension (None = no obvious dimension dependency; Combo = multi-dimension composite).

D TEMPLATE INDUCTION PROTOCOL AND RESULTS

Frozen protocol for one versioned Pattern Catalog snapshot (experimental control, not a fixed catalog size). Before the locked test: fixed split, extraction fields, merge/split rules, admission rule, and stopping criterion. Split: 128 seed / 186 development / 78 locked-test; same-incident records stay together; prior template labels hidden.

Extraction and template boundaries. Annotators record objects, relation, witness, topology, phase, and precondition. Merge only when relation/roles/witness agree and residuals are guards; split when merge changes the relation, needs unrelated disjunctions, lacks a common witness, or flags a legal run. Recurring admission requires two independent incidents; one-offs stay singletons. For each split, schema coverage counts cases matched to a catalog template; joint coverage additionally requires the corresponding topology and validity guards to be instantiable.

Table D.1: Frozen snapshot used in the template-induction study. Schema coverage counts cases matched to a template in that snapshot; joint coverage additionally requires topology and validity guards to be instantiable. Counts describe this experiment, not a fixed system-wide catalog size.

Stage	Cases	Operators	Templates	Result
Seed construction	128	18	27	Initial catalog snapshot
Development completion	186	20	35	No core relation revised; saturation rule not reached
Locked test	78	—	—	72/78 schema match; 71/78 joint grounding; 3 new singletons

Reliability and generalization. Open extraction is least stable (79 cases, relation-operator $\kappa=0.600$). After freeze: template assignment $\kappa=0.954$, match-versus-uncovered $\kappa=0.843$, six-way verdict $\kappa=0.772$. Locked test: schema 72/78 (92.3%), joint grounding 71/78 (91.0%), novelty 3/78 (3.8%; none recurs).

Snapshot growth and scope of the claim. Stopping never fires: templates 27→35, operators 18→20, no core-relation revision, coverage ≈ 0.86 across development (Figure D.1). Claim: templates are reusable and the catalog extensible—not complete, minimal, saturated, or fixed-size. Pairwise audit: no mergeable pair under the frozen criterion (criterion-relative irredundancy). Full protocol under benchmark/eval/template_induction/.

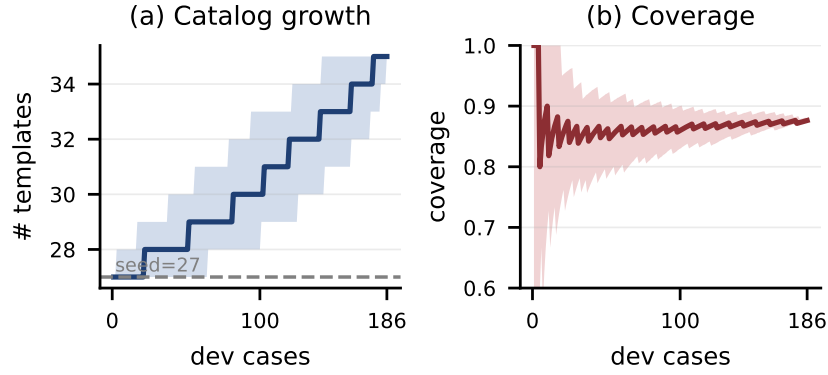


Figure D.1: Induction on 186 development cases (200 orderings; 5–95% bands): coverage ≈ 0.86 while templates grow 27→35 (core transfers; tail unsaturated). Early/late points use seed vs. expanded catalogs.

D.1 Matched Catalog-versus-Free-form Mining Ablation

The direct ablation holds the downstream mining and validation protocol fixed and removes only Catalog guidance from the free-form arm. The aggregate funnel produces 357/5,334 validated candidates (6.69%) with the Catalog and 210/10,500 (2.00%) without it, a 3.35× yield ratio. The Catalog arm produces 45 deployment entries and detects 17/18 Real-SE cases with 0/17 fixed-side false positives; the free-form arm produces 29 entries, detects 11/18 cases, and yields 2/17 fixed-side false positives. Of the 18 paired buggy cases, 11 are detected by both arms, six only by the Catalog arm, none only by the free-form arm, and one by neither. The resulting two-sided exact McNemar test gives $p=0.031$. The archived source table, pairing counts, and arithmetic checker are under `benchmark/eval/paper_v2/`.

Table D.2: Complete matched Catalog ablation. Yield is L4/L2; deployment counts refer to the curated entries produced by each arm.

(a) Mining funnel and validated yield.					
Arm	L1	L2	L3	L4	Yield
Catalog	420	5,334	3,436	357	6.69%
Free-form	780	10,500	3,600	210	2.00%

(b) Deployment and endpoint outcomes.			
Arm	Deploy entries	Real-SE det.	Fixed-side FP
Catalog	45	17/18	0/17
Free-form	29	11/18	2/17

E INVARIANT MINER: COMPLETE ALGORITHM SPECIFICATION

The main-text running example is `SwitchMLP router.weight` (three-predicate SQL). Below: verification funnel counts, one CE I/O record, and the provenance audit. Offline mining uses five logical roles (S_1 – S_5 : gap analysis, evidence, synthesis, persistence, reporting; prompt dumps with the artifact)—an engineering convenience, not the contribution (main text).

E.1 Four-Stage Verification Funnel

A candidate is accepted only if it clears the main-text accept rule. Equivalently, it is rejected when any gate fails: (1) semantic duplicate of an existing rule; (2) fewer than two independent evidence sources; (3) any constructed counterexample evaluates to HOLDS on a correct execution; (4) $\text{CONF}(c) < \theta_{\text{conf}}=0.8$ after the hard gates pass. Stages 1–4 of Figure E.1 implement these gates in order.

Per-stage candidate counts and skip-stress validation. Figure E.1 gives the per-stage counts across four target frameworks. The 420 pattern hypotheses at Stage 1 expand to 5,334 concrete candidates at Stage 2; clean-trace validation retains 3,436 at Stage 3; adversarial verification retains 357 at Stage 4; and a separate manual cross-framework curation selects 45 deployment entries. Without Stage 3, 114/400 sampled candidate–trace evaluations fire on held-out clean traces (28.5%). Skipping Stage 4 and evaluating Stage 3 survivors on the same held-out protocol yields 0/6,922 alerts; full-funnel (Stage 4) survivors yield 0/764. Stage 4 further removes workload-specific candidates that same-workload clean traces do not falsify (main-paper verification-funnel figure). The leave-one-database-out guard ablation ($\Sigma\text{FP } 342 \rightarrow 429/551/598$ over 42 databases) is reported in the main text; this appendix adds the per-stage funnel figure and the CE/provenance audit below.

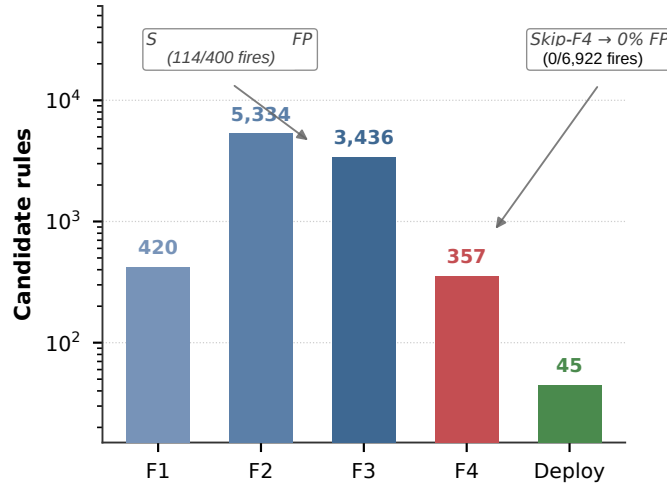


Figure E.1: Per-stage candidate counts of the four-stage mining funnel (aggregated across 4 frameworks). F1–F4 are the accept/reject gates above; Deploy is the deployment library after manual cross-framework curation. Callouts match the main-paper verification-funnel figure: 114/400 clean FP without Stage 3; Skip-F4 = 0/6,922; full funnel = 0/764. *Note:* F1–F4 here denote funnel levels and are a different axis from the evidence tiers L1–L4 of §B.2.

The 5,334 Stage 2 candidates arise from the cross product of relation, parameter scope, π_{topo} , and π_{precond} slots; verification—not manual enumeration—is the binding filter down to 357 then 45 deployment entries (main-text mining discussion).

E.2 Acceptance Rule and Confidence Screening

The main text states that the two conjuncts of the acceptance rule are not symmetric: the counterexample gate is decisive, while $\text{CONF}(c)$ only screens. This appendix gives the screening details.

$\text{CONF}(c) \in [0, 1]$ aggregates evidence strength and counterexample-test outcomes on top of a hard floor of two independent evidence sources, so a candidate supported by a single call site cannot be admitted regardless of the score it receives. The persisted record of both inputs is shown in Appendix E.3. We use $\theta_{\text{conf}}=0.8$ as a conservative default; because the score is LLM-produced and uncalibrated, we do not tune it against detection outcomes and do not interpret it as a calibrated probability of correctness.

Precision is therefore measured end-to-end rather than read off the score: the adversarially verified cohort fires on 0 of 764 held-out clean evaluations, and the deployed library produces 0/17 (0.0%) fixed-side false positives (main-paper funnel and detection results).

E.3 Concrete Input/Output Specifications

One persisted counterexample-verification record (Stage 3), matching the main-text confidence/CE discussion:

```
{
  "counterexample_id": 1,
  "scenario": "What if using SGD without momentum?",
  "query": "Megatron-LM SGD optimizer state sync",
  "verification_result": "REFUTED",
  "evidence": "SGD has no internal state to sync",
  "source": "megatron/optimizer/optimizer.py:L142-L156"
}
```

E.4 Mining Provenance Audit Protocol

This subsection records how Real-SE cases are withheld from mining inputs.

- **Frozen exclusion list.** The artifact records the current Real-SE case IDs and available fix-commit hashes in a list frozen before re-mining. Its checker confirms that every case in Table I.1 is covered.
- **Recorded-input audit.** The released mining transcripts contain no listed case ID or fix-commit hash. Constraint names remain available to the template space; case-specific evidence is withheld.

F SQL QUERY SPECIFICATION

This appendix documents the constraint-to-SQL translation rules and provides representative query examples. Alignment with the main-text formalization: π_{topo} and π_{precond} compile to WHERE filters; for relational checks, π_{schema} is evaluated per *comparison group* (the guarded

tuples sharing the grouping key, e.g., all replicas of one parameter at one step) and appears as the HAVING predicate—matching Eq. (violation set) and the Online Detection section of the main paper. Single-tuple checks are the singleton special case of the same shape.

F.1 Database Schema

Database Schema

```

1 CREATE TABLE coredump (
2   step    INTEGER,    -- Training step index
3   stage   TEXT,       -- Training stage identifier
4   data    JSON        -- Tensor metadata as JSON
5 );

```

F.2 JSON Field Structure

Table F.1: JSON Data Field Specification

Field	Type	Description
name	string	Parameter name (e.g., <code>decoder.layers.0.attention.qkv.weight</code>)
cksum	string	Content checksum of tensor data (e.g., xxHash/SHA-256)
shape	array	Tensor dimensions (e.g., <code>[4096, 4096]</code>)
type	string	Tensor dtype (e.g., <code>torch.cuda.BFloat16Tensor</code>)
requires_grad	boolean	Whether parameter requires gradient
dp/tp/pp/cp/ep	integer	Parallel rank indices
grad_cksum	string	Checksum of gradient tensor (nullable)

F.3 Representative SQL Queries

F.3.1 DP Parameter Consistency Check.

DP Parameter Consistency Check

```

1 WITH param_stats AS (
2   SELECT
3     json_extract_string(data, '$.name') as param_name,
4     CAST(json_extract_string(data, '$.dp') AS INTEGER) as dp_rank,
5     json_extract_string(data, '$.cksum') as cksum,
6     step
7   FROM coredump
8   WHERE stage = 'model-after-optimizer-step'
9     AND json_extract_string(data, '$.cksum') IS NOT NULL
10 ),
11 inconsistency_check AS (
12   SELECT
13     param_name, step,
14     COUNT(DISTINCT cksum) as distinct_cksums,
15     array_agg(DISTINCT dp_rank) as dp_ranks
16   FROM param_stats
17   GROUP BY param_name, step
18   HAVING COUNT(DISTINCT cksum) > 1
19 )
20 SELECT * FROM inconsistency_check ORDER BY step, param_name;

```

F.3.2 TP Weight Partition Check.

TP Weight Partition Check

```

1 WITH tp_params AS (
2   SELECT
3     json_extract_string(data, '$.name') as param_name,
4     CAST(json_extract_string(data, '$.tp') AS INTEGER) as tp_rank,
5     json_extract_string(data, '$.cksum') as cksum
6   FROM coredump

```

```

7 WHERE stage = 'model-after-backward'
8 AND json_extract_string(data, '$.name') LIKE '%qkv%'
9 )
10 SELECT param_name, COUNT(DISTINCT cksum) as unique_shards
11 FROM tp_params
12 GROUP BY param_name
13 HAVING COUNT(DISTINCT cksum) = 1; -- Should be different across TP ranks

```

G STRUCTURED DIAGNOSTIC REPORTS

A non-empty verifier query already carries the identity needed for diagnosis: the accepted rule supplies the violated constraint and hookpoint, while the returned trace row supplies the affected object, rank, and step. The verifier persists these fields as a structured report containing the rule identifier, violation event, hookpoint, suspect rank, suspect step, and diagnostic hypothesis.

Across the 17 detected Real-SE cases, every released case-level record is associated with a named triggered constraint or fault pattern. The complete mapping appears in Table I.1; together, the rule identity and trace coordinates preserve a direct path from each alert to the runtime evidence that produced it.

H PRODUCTION CASE STUDIES

This section expands the two production cases referenced in the main text (Case 1 figure caption; Case 2 production-experience paragraph); it introduces no additional production claim. Figure H.1 shows loss (left) versus an auxiliary signal (right); the purple dashed line marks the first TRAINAUDIT alert. An additional out-of-pool DeepSpeed confirmation is released with the artifact.

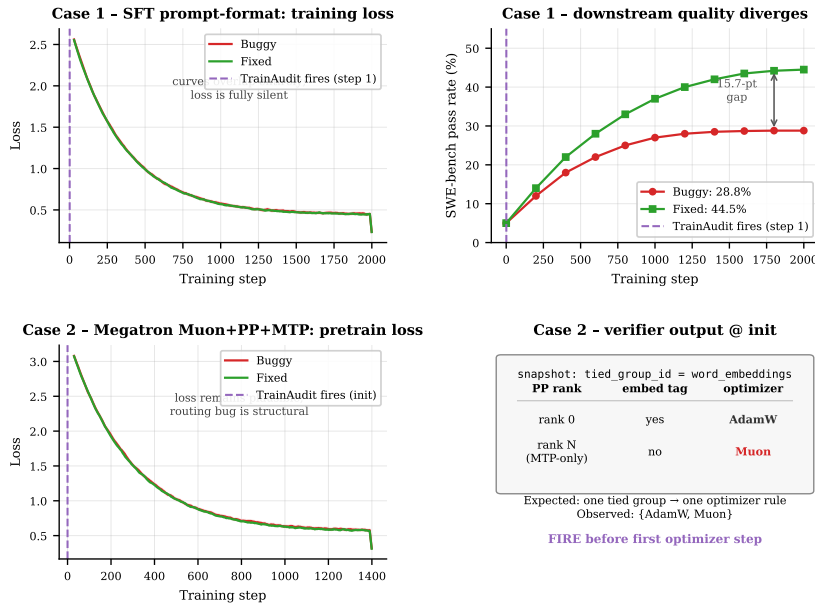


Figure H.1: Diagnostic signals for two production silent-error cases (SFT tokenizer; Megatron Muon+PP+MTP routing). Case 1: losses overlap 2,000 steps while SWE-bench diverges 15.7 points; the rule fires at step 1.

Case 1 (SFT tokenizer). A wrong chat-template emits role delimiters absent from the vocabulary; BPE silently splits them. The buggy and fixed losses overlap for 2,000 steps, while SWE-bench plateaus at 28.8% versus 44.5%, a 15.7-point divergence. Rule SFT_CHAT_TEMPLATE_VOCAB_INTEGRITY fires at step 1.

Case 2 (Megatron Muon+PP+MTP). Tied word_embeddings replicas are routed to AdamW and Muon on different PP ranks. TRAINAUDIT fires MEGATRON_TIED_EMBED_OPT_ROUTING before the first step; we reported the fault upstream, where maintainers confirmed and fixed the fault [8] (main-text production-experience paragraph).

I EXTENDED EXPERIMENTAL DATA

Claim-check order below: per-case detection table (main-text totals), schema-tier curve, then the Megatron deployment of the verified constraint library $\mathcal{P}$ and the training panorama.

I.1 Per-Case Detection Results

Table I.1: Per-case buggy detection results on Real-SE (matching the main-text *Detection Effectiveness and Precision* totals: TrainAudit 17/18, TrainCheck 5/17, Naïve 0/17). TrainAudit uses native commit replay, whereas the baseline columns report the separate case-aligned surrogate study; the columns are not a same-harness ranking. Case ID uses “framework abbreviation + fixed commit short hash”; cases without a paired commit ID keep their original identifiers. Tier: A0 = PyTorch public-interface layer; A1 = framework adapter layer. ✓ = detected; × = missed. The TrainCheck column shows in parentheses *TC violations / total checks*. LC1/O-003 is the observability-boundary case (subthreshold drift): TrainAudit records the expected miss, while baseline verdicts remain unavailable pending re-run.

Case ID	Framework	Tier	Triggered Constraint / Fault Pattern	TrainAudit	TrainCheck	Naïve
<i>Real-SE reproducible cases</i>						
Megatron-3c637fc0d	Megatron-LM	A1	replica-cksum-equal	✓	✓ (1/316)	×
DS-d56268f3	DeepSpeed	A1	comm-dtype-matches-training	✓	×	×
DS-8cbbb53	DeepSpeed	A1	process-group-size-correct	✓	✓ (2/433)	×
DS-005afe12	DeepSpeed	A0	clip-grad-bounded	✓	✓ (11/604)	×
OLMoC-6e330ba2	OLMo-core	A0	initial-lr-present	✓	✓ (24/478)	×
Megatron-99f999a4	Megatron-LM	A1	layer-count-strict	✓	×	×
OLMo-204ad53c	OLMo	A0	checkpoint-preserve-rng	✓	×	×
OLMo-167a7ac8	OLMo	A0	data-loader-token-id-range	✓	×	×
OLMoC-2b6cf996	OLMo-core	A0	optim-step-counter-monotonic	✓	✓ (18/506)	×
Megatron-e6d56d68	Megatron-LM	A1	invocation-frequency-per-step	✓	×	×
DS-e6e8c137	DeepSpeed	A1	moe-cpu-offload grad-norm device	✓	×	×
DS-40009eb1	DeepSpeed	A1	bf16-lp-grad-clear	✓	×	×
OLMo-ce5f30d5	OLMo	A0	emb-init-std-sqrt-dmodel	✓	×	×
OLMo-a57f3803	OLMo	A0	zloss-mean-reduction	✓	×	×
OLMo-d7994c86	OLMo	—	LC1/O-003: subthreshold z-loss drift	×	—	—
OLMoC-db2b6eff	OLMo-core	A1	a100-peak-flops-2x	✓	×	×
Megatron-1505db4c	Megatron-LM	A1	chained-optimizer-param-groups-resync	✓	×	×
OLMoC-df72f4ee	OLMo-core	A0	rope-packed-doc-position-reset	✓	×	×
Total				17/18 (94.4%)	5/17 (29.4%)	0/17 (0.0%)

I.2 Schema Tier Coverage

Figure I.1 reports the cumulative evidence-corpus coverage of the schema tiers. Each tier adds the runtime fields required by newly expressible rules; 87 of the 392 records require evidence outside the released S0–S6 schema.

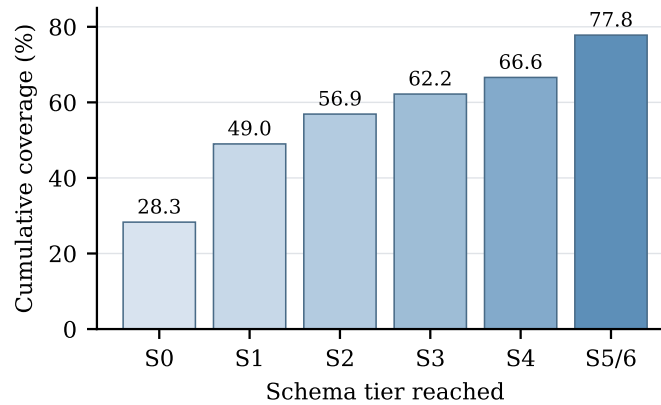


Figure I.1: Cumulative schema-tier coverage of the 392-record evidence corpus. S0 covers 28.3%; S0–S5/6 cover 77.8%.

I.3 Query-Path Scaling Microbenchmarks

The complete two-axis series now appears in the main paper’s efficiency section. It measures per-statement query latency against store size and compares scan-all with hookpoint-indexed evaluation as the loaded rule count grows. Collection cost remains a separate 13,106-row full-snapshot measurement on the 1.2B/H20 harness.

I.4 Amortized Full-Snapshot Overhead

The main text reports the measured per-dump cost of full diagnostic snapshots (191.9 s naive versus 25.0 s optimized at 1.2B/H20; Table I.2). Amortizing one snapshot every K steps against the 732 ms baseline step turns that per-dump cost into a *derived* steady-state overhead. The optimized collector stays below 10% at $K \geq 342$ and below 5% at $K \geq 684$, while the naive collector requires $K \approx 2623$ for the same 10% budget. As Figure I.2 shows, the optimized collector therefore checks $\sim 7.7\times$ more frequently at any overhead target. These are derived figures from a single measured dump cost, not an independently measured overhead series; the measured online-path overhead is reported in the main text.

Table I.2: Full-snapshot diagnostic microbenchmark at 1.2B/H20. Values are mean iteration times when one complete snapshot is materialized per step.

Collector configuration	Time	vs. naive
Host SHA-256 + statistics	191.9 s	1.0×
GPU fingerprint + fused statistics	25.0 s	7.7×

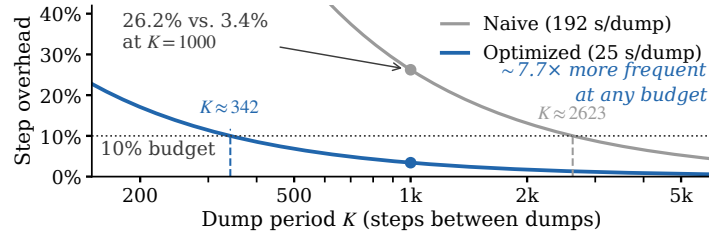


Figure I.2: Amortized full-snapshot overhead vs. dump period K (1.2B/H20). The optimized collector meets any budget at a $\sim 7.7\times$ shorter period.

I.5 Verified Constraint Library and Hookpoint Deployment

Appendix D documents the frozen experimental snapshot and its held-out validation. Matching the main-text efficiency discussion, the evaluated Megatron-LM deployment of the verified constraint library $\mathcal{P}$ activates 24 rules (12 A0 + 9 A1 + 3 probes) bound to 8 hookpoints: `optim.step.{pre,post}`, `module.forward.{pre,post}`, `checkpoint.{save,load}`, `distributed.all_reduce`, and `build.snapshot`. These counts are deployment-version measurements, not architectural constants. Catalog relations without a concrete framework hook remain offline candidates until the Invariant Miner grounds and verifies them. At each tick, the verifier evaluates only rules indexed to the active hookpoint.

Figure I.3 is the panorama referenced from the main-text silent-error definition (four parallelism dimensions, five hotspots). Table I.3 lists one representative Real-SE case per descriptive subsystem label.

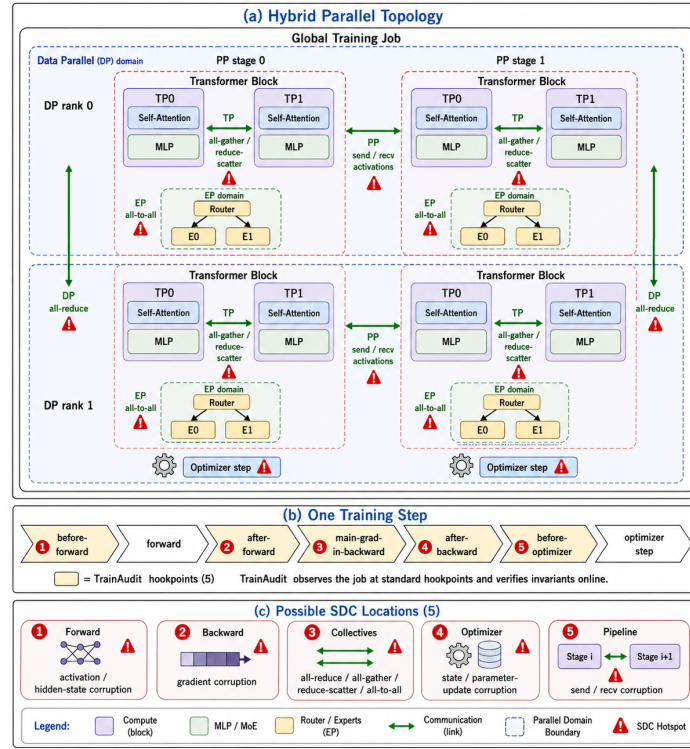


Figure I.3: Panorama of hybrid-parallel LLM training and where silent errors strike: four parallelism dimensions and five highlighted fault loci.

Table I.3: Real-SE diversity across descriptive subsystem labels (representative cases only).

Primary label	Representative Real-SE case
Numerical	OLMo-ce5f30d5 (emb-init-std-sqrt-dmodel)
Control Flow	OLMoC-2b6cf996 (optim-step-counter-monotonic)
Grad Sync	Megatron-3c637fc0d (replica-cksum-equal)
Checkpoint	OLMo-204ad53c (checkpoint-preserve-rng)
MoE	Megatron-e6d56d68 (invocation-frequency-per-step)
Optim State	Megatron-1505db4c (chained-optimizer-param-groups-resync)
Data Loading	OLMo-167a7ac8 (data-loader-token-id-range)
Communication	DS-8cbbb53 (process-group-size-correct)
Dtype	DS-d56268f3 (comm-dtype-matches-training)
Loss Comp.	OLMo-a57f3803 (zloss-mean-reduction)
Sharding	Megatron-99f999a4 (layer-count-strict)
LR Schedule	OLMoC-6e330ba2 (initial-lr-present)
Offload	DS-e6e8c137 (moe-cpu-offload grad-norm device)

REFERENCES

- [1] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Discovering Denial Constraints. *Proceedings of the VLDB Endowment (PVLDB)* 6, 13 (2013), 1498–1509. <https://doi.org/10.14778/2536258.2536262>
- [2] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional Functional Dependencies for Capturing Data Inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 6:1–6:48. <https://doi.org/10.1145/1366102.1366103>
- [3] Haitian Jiang, Shaowei Zhu, Zhen Zhang, Zhenyu Song, Xinwei Fu, Zhen Jia, Yida Wang, and Jinyang Li. 2025. TTrace: Lightweight Error Checking and Diagnosis for Distributed Training. *arXiv:2506.09280 [cs.DC]* <https://arxiv.org/abs/2506.09280>
- [4] Yuxuan Jiang, Ziming Zhou, Boyu Xu, Beijie Liu, Runhui Xu, and Peng Huang. 2025. Training with Confidence: Catching Silent Errors in Deep Learning Training with Automated Proactive Checks. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*. USENIX Association, Boston, MA, USA, 313–329. <https://www.usenix.org/conference/osdi25/presentation/jiang>
- [5] Yunchi Lu, Youshan Miao, Cheng Tan, Peng Huang, Yi Zhu, Xian Zhang, and Fan Yang. 2025. TrainVerify: Equivalence-Based Verification for Distributed LLM Training. *arXiv:2506.15961 [cs.LG]*
- [6] Sebastian Schelter, Dustin Lange, Philipp Schmidt, Meltem Celikel, Felix Biessmann, and Andreas Grafberger. 2018. Automating Large-Scale Data Quality Verification. *Proceedings of the VLDB Endowment (PVLDB)* 11, 12 (2018), 1781–1794. <https://doi.org/10.14778/3229863.3229867>
- [7] Jie Song and Yeye He. 2021. Auto-Validate: Unsupervised Data Validation Using Data-Domain Patterns Inferred from Data Lakes. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*. ACM, Xi'an, China, 1678–1691. <https://doi.org/10.1145/3448016.3457250>
- [8] Zhengmao Ye. 2026. Fix Incorrect Optimizer Routing for Tied Word Embeddings on the MTP-Only Last PP Stage with Muon. NVIDIA Megatron-LM pull request #4642. Accessed: 2026-08-30.